

Deep Fake Audio Detection Using CNN-LSTM

Author:
Omer Farooq Khan

May 11, 2026

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Problem Statement	3
2	Dataset Preparation	4
2.1	Dataset Structure	4
2.2	Manifest Generation	4
2.2.1	Label Mapping	4
3	Audio Preprocessing	5
3.1	Preprocessing Pipeline	5
3.2	Why Mono Conversion?	5
3.3	Why 16 kHz Sampling Rate?	5
3.4	Why Fixed 2-Second Clips?	5
3.5	MFCC Feature Extraction	5
4	Model Architecture	6
4.1	CNN-LSTM Hybrid Model	6
4.1.1	CNN Component	6
4.1.2	LSTM Component	6
4.2	Output Layer	6
4.3	Loss Function	7
5	Training Pipeline	8
5.1	Training Process	8
5.2	Evaluation Metrics	8
5.3	Checkpointing	8
6	Inference System	9
6.1	Gradio Interface	9
6.2	Chunking Strategy	9
6.2.1	Why 50% Overlap?	9
6.3	Prediction Rule	9
7	Visualizations	10
7.1	Real Audio Visualizations	10
7.1.1	Waveform	10
7.1.2	Spectrogram	10
7.1.3	Mel Spectrogram	11

7.1.4	MFCC	11
7.2	Fake Audio Visualizations	11
7.2.1	Waveform	11
7.2.2	Spectrogram	12
7.2.3	Mel Spectrogram	12
7.2.4	MFCC	12
8	Performance Evaluation	13
8.1	Dataset Statistics	13
8.2	Training Curves	14
8.2.1	Training Loss	14
8.2.2	Training Accuracy	14
8.2.3	Validation AUC	15
8.3	Confusion Matrix	15
8.4	Precision-Recall Curve	16
9	Results	17
9.1	Final Metrics	17
10	Limitations	18
11	Conclusion	19

Chapter 1

Introduction

1.1 Project Overview

The objective of this project is to classify audio clips as either real or fake using a supervised deep learning pipeline. The system processes audio samples, extracts MFCC features, trains a hybrid CNN-LSTM model, evaluates performance using multiple metrics, and provides a Gradio-based user interface for inference.

The project workflow consists of:

- Building a dataset manifest
- Audio preprocessing and MFCC extraction
- CNN-LSTM model training
- Evaluation and visualization
- Gradio deployment for inference

1.2 Problem Statement

Deepfake audio and synthetic speech generation technologies have become increasingly realistic. Detecting whether an audio clip is genuine or AI generated is an important cybersecurity and information security challenge.

This project aims to develop an audio classification system capable of distinguishing fake voice recordings from real human speech.

Chapter 2

Dataset Preparation

2.1 Dataset Structure

The dataset contains two folders:

- `real/`
- `fake/`

Each folder contains WAV audio clips used for supervised learning.

2.2 Manifest Generation

A Python script named `prepare_manifest.py` scans the dataset and creates a CSV manifest file.

2.2.1 Label Mapping

Class	Label
Real	0
Fake	1

Table 2.1: Binary Label Mapping

The generated CSV contains:

- Filepath
- Label

Chapter 3

Audio Preprocessing

3.1 Preprocessing Pipeline

The preprocessing stage ensures all audio samples have a consistent format before feature extraction.

The following steps are applied:

1. Convert audio to mono
2. Resample audio to 16 kHz
3. Pad or trim audio to 2 seconds
4. Extract MFCC features

3.2 Why Mono Conversion?

Stereo information is not important for distinguishing fake and real speech. Converting to mono reduces computational complexity and standardizes inputs.

3.3 Why 16 kHz Sampling Rate?

16 kHz preserves speech-relevant frequencies while reducing memory and computation requirements.

3.4 Why Fixed 2-Second Clips?

Deep learning models require fixed-size tensors for stable batching and training.

3.5 MFCC Feature Extraction

The project uses 40 MFCC coefficients.

MFCCs provide a compact representation of speech timbre and spectral characteristics, which are useful for detecting synthetic audio artifacts.

Chapter 4

Model Architecture

4.1 CNN-LSTM Hybrid Model

The model combines convolutional layers with recurrent layers.

4.1.1 CNN Component

The CNN extracts local spectral patterns from MFCC feature maps.

It helps identify:

- Spectral artifacts
- Frequency inconsistencies
- Texture patterns

4.1.2 LSTM Component

The LSTM captures temporal dependencies across time frames.

This helps model:

- Speech continuity
- Temporal coherence
- Dynamic transitions

4.2 Output Layer

The model outputs a single logit for binary classification.

Sigmoid activation converts the logit into:

$$P(\text{fake})$$

4.3 Loss Function

Binary Cross Entropy with Logits Loss:

$$\mathcal{L} = -(y \log(\sigma(x)) + (1 - y) \log(1 - \sigma(x)))$$

This combines sigmoid activation and BCE loss in a numerically stable manner.

Chapter 5

Training Pipeline

5.1 Training Process

The dataset is divided into:

- Training Set
- Validation Set
- Test Set

A stratified split is used to preserve class balance.

5.2 Evaluation Metrics

The following metrics are used:

- Accuracy
- F1 Score
- ROC-AUC

5.3 Checkpointing

The best model checkpoint is saved as:

`best.pth`

Chapter 6

Inference System

6.1 Gradio Interface

A Gradio web application is used for inference.

Users can upload audio files for prediction.

6.2 Chunking Strategy

The model was trained on 2-second clips, so longer audio files are divided into overlapping chunks.

6.2.1 Why 50% Overlap?

Overlap reduces boundary artifacts and improves prediction stability.

6.3 Prediction Rule

Average probabilities from all chunks are computed.

Decision rule:

Fake if $P > 0.7$

The threshold was intentionally increased to reduce false positives.

Chapter 7

Visualizations

7.1 Real Audio Visualizations

7.1.1 Waveform

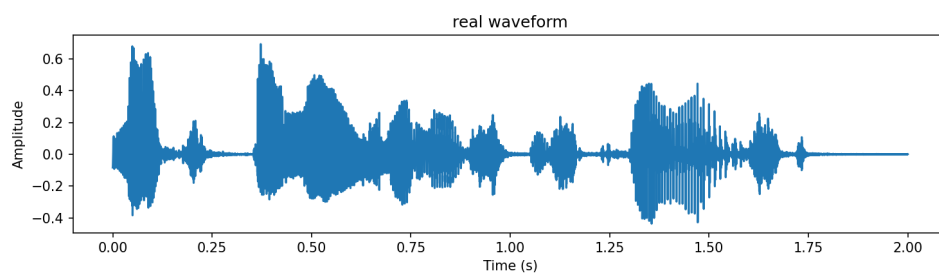


Figure 7.1: Real Audio Waveform

7.1.2 Spectrogram

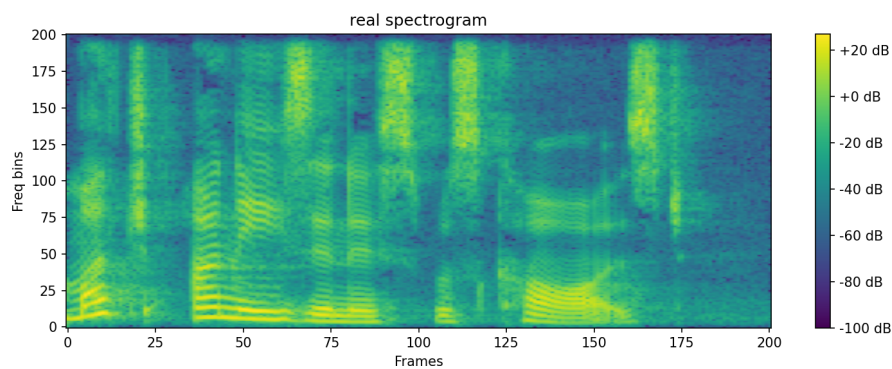


Figure 7.2: Real Audio Spectrogram

7.1.3 Mel Spectrogram

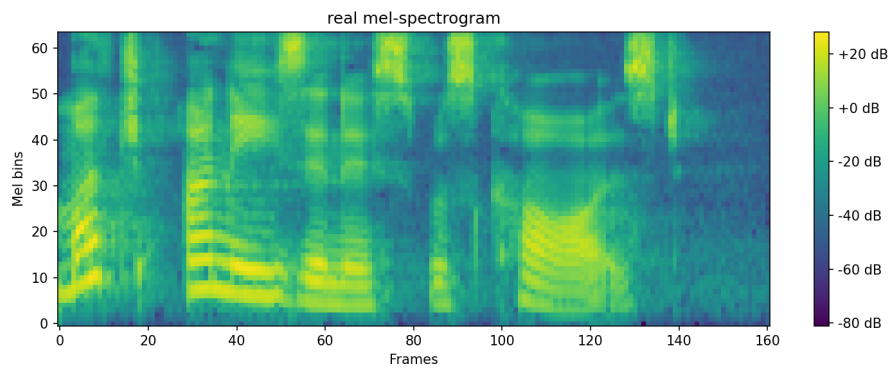


Figure 7.3: Real Audio Mel Spectrogram

7.1.4 MFCC

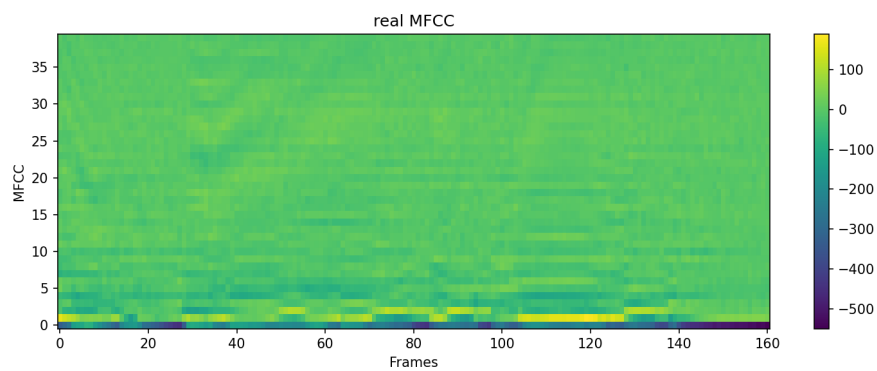


Figure 7.4: Real Audio MFCC

7.2 Fake Audio Visualizations

7.2.1 Waveform

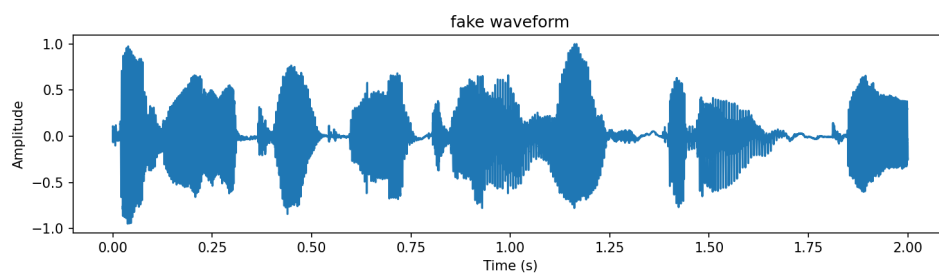


Figure 7.5: Fake Audio Waveform

7.2.2 Spectrogram

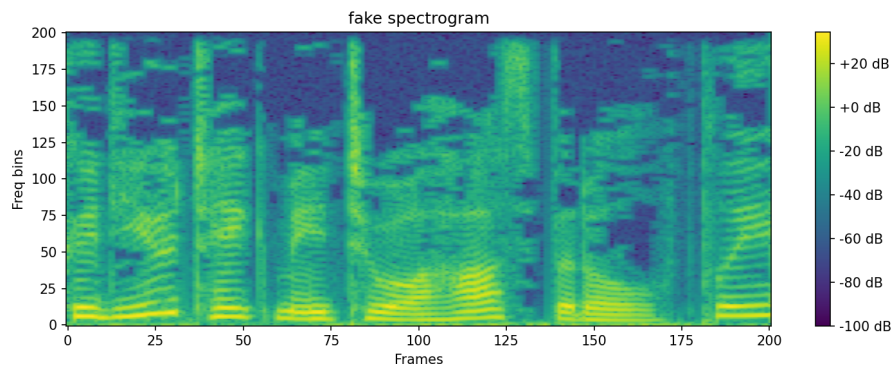


Figure 7.6: Fake Audio Spectrogram

7.2.3 Mel Spectrogram

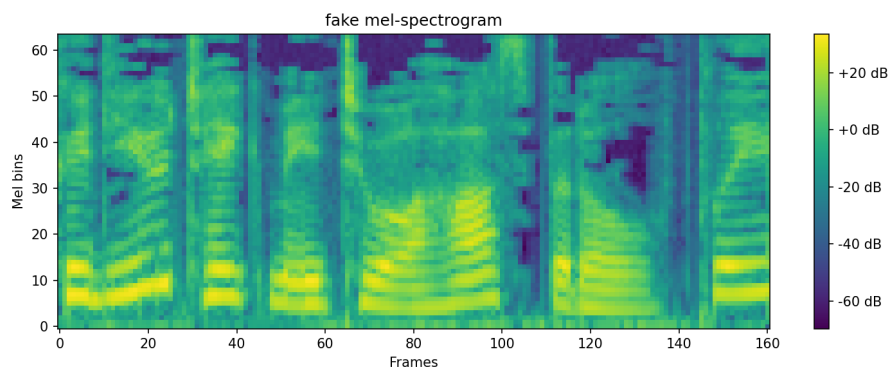


Figure 7.7: Fake Audio Mel Spectrogram

7.2.4 MFCC

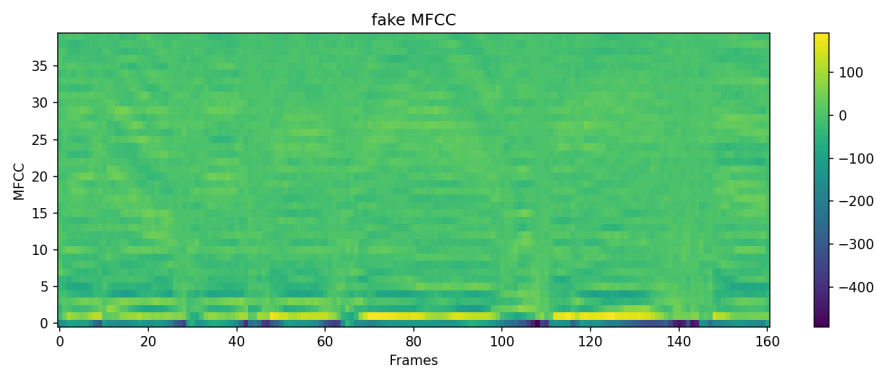


Figure 7.8: Fake Audio MFCC

Chapter 8

Performance Evaluation

8.1 Dataset Statistics

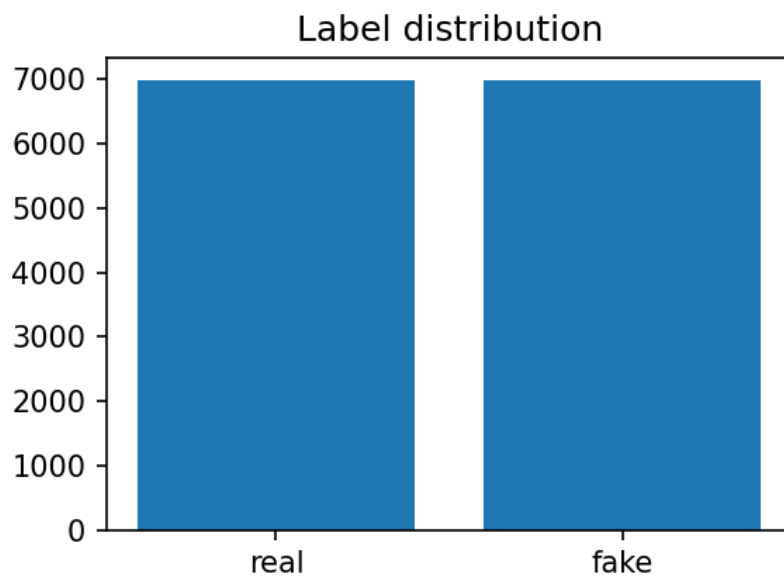


Figure 8.1: Dataset Label Distribution

8.2 Training Curves

8.2.1 Training Loss

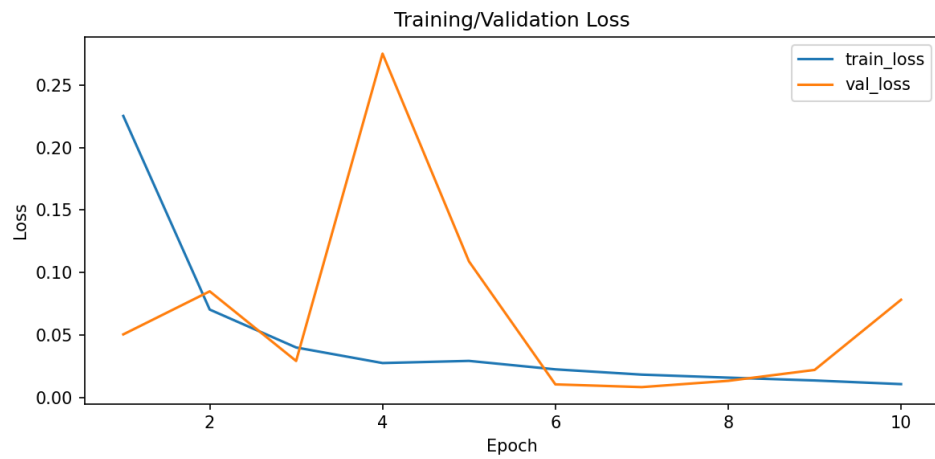


Figure 8.2: Training Loss Curve

8.2.2 Training Accuracy

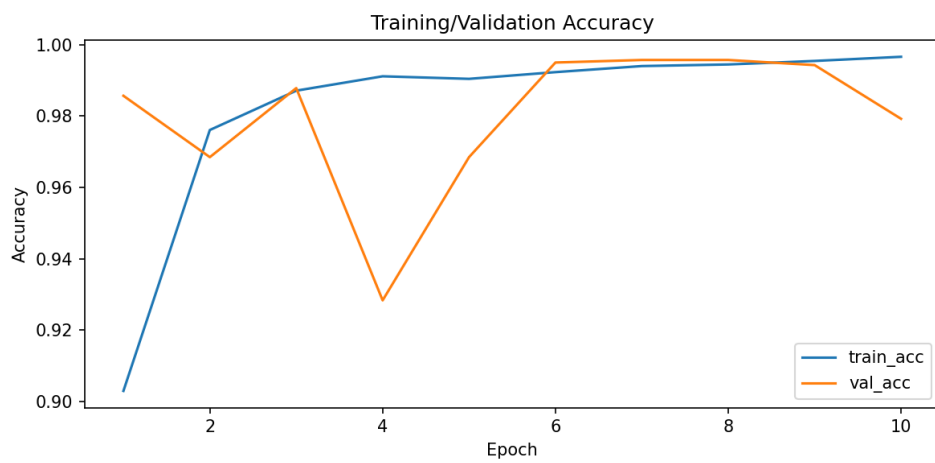


Figure 8.3: Training Accuracy Curve

8.2.3 Validation AUC

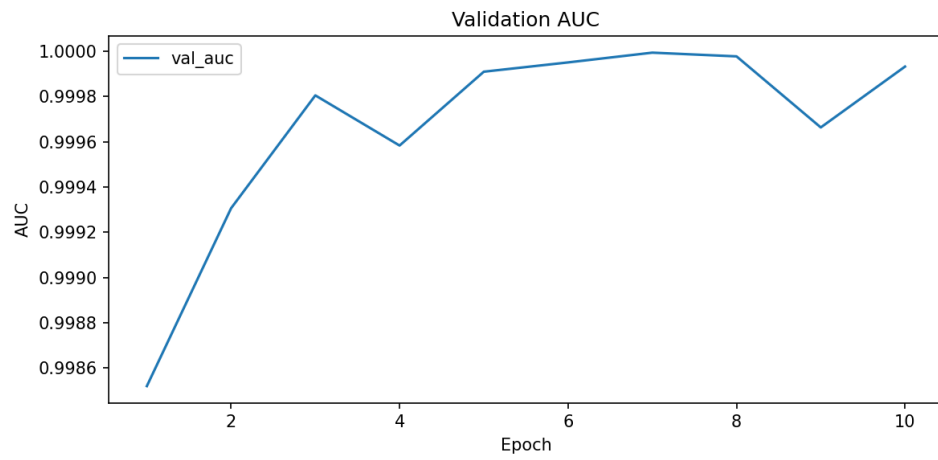


Figure 8.4: Validation AUC

8.3 Confusion Matrix

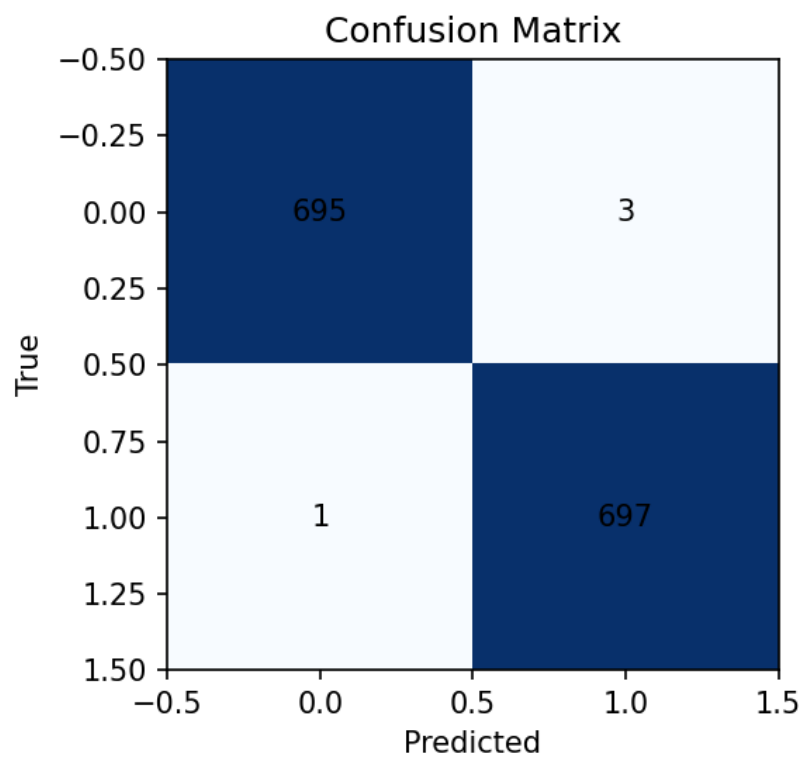


Figure 8.5: Confusion Matrix

8.4 Precision-Recall Curve

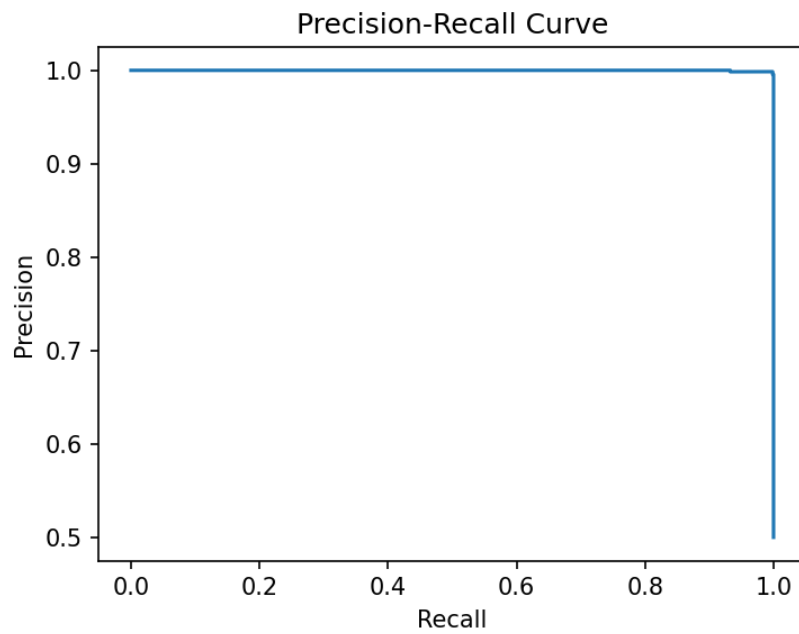


Figure 8.6: Precision-Recall Curve

Chapter 9

Results

9.1 Final Metrics

Metric	Value
Test Loss	0.0084
Accuracy	0.9971
F1 Score	0.9971
ROC-AUC	0.9999

Table 9.1: Final Evaluation Metrics

The model achieved very high classification performance on the test split.

Chapter 10

Limitations

- Results are based on the current dataset split only.
- Dataset artifacts may artificially increase performance.
- Generalization to unseen real-world fake audio is not guaranteed.
- Out-of-distribution testing is still required.

Chapter 11

Conclusion

This project successfully implemented a deep learning pipeline for fake vs real voice detection using MFCC features and a CNN-LSTM architecture.

The model achieved excellent performance on the provided dataset while maintaining a simple and deployable inference workflow through Gradio.

The project demonstrates how deep learning can be applied to information security problems involving synthetic media detection.

References

1. Research papers on Deepfake Audio Detection